



Open-Source AI Cookbook documentation

Using LLM-as-a-judge 🧑 for an automated and versatile evaluation ▾



Join the Hugging Face community

and get access to the augmented documentation experience

[Sign Up](#)

to get started



Using LLM-as-a-judge 🧑 for an automated and versatile evaluation

[Copy page](#)

Authored by: [Aymeric Roucher](#)

Evaluation of Large language models (LLMs) is often a difficult endeavour: given their broad capabilities, the tasks given to them often should be judged on requirements that would be very broad, and loosely-defined. For instance, an assistant's answer to a question can be:

- not grounded in context
- repetitive, repetitive, repetitive
- grammatically incorrects
- Excessively lengthy and characterized by an overabundance of words, leading to a situation where the discourse or written content becomes overly detailed and protracted
- incoherent
- ...

The list of criteria goes on and on. And even if we had a limited list, each of these would be hard to measure: “devising a rule-based program to assess the outputs is extremely challenging.

[Ask HuggingChat](#)

Traditional evaluation metrics based on the similarity between outputs and reference answers (e.g., ROUGE, BLEU) are also ineffective for these questions.”

✅ A powerful solution to assess outputs in a human way, without requiring costly human time, is LLM-as-a-judge. This method was introduced in [Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena](#) - which I encourage you to read.

💡 The idea is simple: ask an LLM to do the grading for you. 🤖✅

But we'll see that it will not work well out-of-the-box: you need to set it up carefully for good results.

```
!pip install huggingface_hub datasets pandas tqdm -q
```

```
import re
import pandas as pd
from tqdm.auto import tqdm
from datasets import load_dataset
from huggingface_hub import InferenceClient, notebook_login

tqdm.pandas() # load tqdm's pandas support
pd.set_option("display.max_colwidth", None)

notebook_login()
```

```
repo_id = "mistralai/Mixtral-8x7B-Instruct-v0.1"

llm_client = InferenceClient(
    model=repo_id,
    timeout=120,
)

# Test your LLM client
llm_client.text_generation(prompt="How are you today?", max_new_tokens=20)
```

1. Prepare the creation and evaluation of our



Let's say you want to give an LLM a specific task, like answering open-ended questions.

The difficulty is that, as we discussed above, measuring the answer's quality is difficult, for instance an exact string match will flag too many correct but differently worded answers as false.

You could get human labellers to judge the outputs, but this is very time-consuming for them, and if you want to update the model or the questions, you have to do it all over again.

✅ In this case you can setup a LLM-as-a-judge.

But to use a LLM-as-a-judge, you will first need to evaluate how reliably it rates your model outputs.

➡ So the first step will be... To create a human evaluation dataset. But you can get human annotations for a few examples only - something like 30 should be enough to get a good idea of the performance. And you will be able to re-use this dataset everytime you want to test your LLM-as-a-judge.

In our case, we will use feedbackQA, which contains 2 human evaluations and scores for each question/answer couple: using a sample of 30 examples will be representative of what your small evaluation dataset could be.

```
ratings = load_dataset("McGill-NLP/feedbackQA")["train"]
ratings = pd.DataFrame(ratings)

ratings["review_1"] = ratings["feedback"].apply(lambda x: x["rating"][0])
ratings["explanation_1"] = ratings["feedback"].apply(lambda x: x["explanation"][0])
ratings["review_2"] = ratings["feedback"].apply(lambda x: x["rating"][1])
ratings["explanation_2"] = ratings["feedback"].apply(lambda x: x["explanation"][1])
ratings = ratings.drop(columns=["feedback"])

# Map scores to numeric values
conversion_dict = {"Excellent": 4, "Acceptable": 3, "Could be Improved": 2, "Bad": 1}
ratings["score_1"] = ratings["review_1"].map(conversion_dict)
ratings["score_2"] = ratings["review_2"].map(conversion_dict)
```



It's always a good idea to compute a baseline for performance: here it can be for instance the agreement between the two human raters, as measured by the Pearson correlation of the scores they give.

```
>>> print("Correlation between 2 human raters:")
>>> print(f"{ratings['score_1'].corr(ratings['score_2'], method='pearson'):.3f}")
```

```
Correlation between 2 human raters:
0.563
```

This correlation between 2 human raters is not that good. If your human ratings are really bad, it probably means the rating criteria are not clear enough.

This means that our “ground truth” contains noise: hence we cannot expect any algorithmic evaluation to come that close to it.

However, we could reduce this noise:

- by taking the average score as our ground truth instead of any single score, we should even out some of the irregularities.
- by only selecting the samples where the human reviewers are in agreement.

Here, we will choose the last option and **only keep examples where the 2 human reviewers are in agreement**.

```
# Sample examples
ratings_where_raters_agree = ratings.loc[ratings["score_1"] == ratings["score_2"]]
examples = ratings_where_raters_agree.groupby("score_1").sample(7, random_state=1214)
examples["human_score"] = examples["score_1"]

# Visualize 1 sample for each score
display(examples.groupby("human_score").first())
```

2. Create our LLM judge



We build our LLM judge with a basic prompt, containing these elements:

- task description
- scale description: minimum, maximum, value types (float here)
- explanation of the output format
- a beginning of an answer, to take the LLM by the hand as far as we can

```
JUDGE_PROMPT = """
You will be given a user_question and system_answer couple.
Your task is to provide a 'total rating' scoring how well the system_answer answers the u
Give your answer as a float on a scale of 0 to 10, where 0 means that the system_answer i

Provide your feedback as follows:

Feedback::
Total rating: (your rating, as a float between 0 and 10)

Now here are the question and answer.

Question: {question}
Answer: {answer}

Feedback::
Total rating: """
```

```
examples["llm_judge"] = examples.progress_apply(
    lambda x: llm_client.text_generation(
        prompt=JUDGE_PROMPT.format(question=x["question"], answer=x["answer"]),
        max_new_tokens=1000,
    ),
    axis=1,
)
```

```
def extract_judge_score(answer: str, split_str: str = "Total rating:") -> int:
    try:
        if split_str in answer:
            rating = answer.split(split_str)[1]
```

```

else:
    rating = answer
    digit_groups = [el.strip() for el in re.findall(r"\d+(?:\.\d+)?", rating)]
    return float(digit_groups[0])
except Exception as e:
    print(e)
    return None

```

```

examples["llm_judge_score"] = examples["llm_judge"].apply(extract_judge_score)
# Rescale the score given by the LLM on the same scale as the human score
examples["llm_judge_score"] = (examples["llm_judge_score"] / 10) + 1

```

```

>>> print("Correlation between LLM-as-a-judge and the human raters:")
>>> print(
...     f"{examples['llm_judge_score'].corr(examples['human_score'], method='pearson'):.3f}"
... )

```

Correlation between LLM-as-a-judge and the human raters:
0.567

This is not bad, given that the Pearson correlation between 2 random, independent variables would be 0!

But we easily can do better. [↑ TOP](#)

3. Improve the LLM judge

As shown by [Aparna Dhinakaran](#), LLMs suck at evaluating outputs in continuous ranges. [This article](#) gives us a few best practices to build a better prompt:

- 🕒 **Leave more time for thought** by adding an Evaluation field before the final answer.
- 📏 **Use a small integer scale** like 1-4 or 1-5 instead of a large float scale as we had previously.
- 🧑 **Provide an indicative scale for guidance.**



- We even add a carrot to motivate the LLM!

```
IMPROVED_JUDGE_PROMPT = """
```

```
You will be given a user_question and system_answer couple.
```

```
Your task is to provide a 'total rating' scoring how well the system_answer answers the u
Give your answer on a scale of 1 to 4, where 1 means that the system_answer is not helpfu
```

```
Here is the scale you should use to build your answer:
```

```
1: The system_answer is terrible: completely irrelevant to the question asked, or very pa
```

```
2: The system_answer is mostly not helpful: misses some key aspects of the question
```

```
3: The system_answer is mostly helpful: provides support, but still could be improved
```

```
4: The system_answer is excellent: relevant, direct, detailed, and addresses all the conc
```

```
Provide your feedback as follows:
```

```
Feedback::
```

```
Evaluation: (your rationale for the rating, as a text)
```

```
Total rating: (your rating, as a number between 1 and 4)
```

```
You MUST provide values for 'Evaluation:' and 'Total rating:' in your answer.
```

```
Now here are the question and answer.
```

```
Question: {question}
```

```
Answer: {answer}
```

```
Provide your feedback. If you give a correct rating, I'll give you 100 H100 GPUs to start
```

```
Feedback::
```

```
Evaluation: """
```

```
examples["llm_judge_improved"] = examples.progress_apply(
    lambda x: llm_client.text_generation(
        prompt=IMPROVED_JUDGE_PROMPT.format(question=x["question"], answer=x["answer"]),
        max_new_tokens=500,
    ),
    axis=1,
)
examples["llm_judge_improved_score"] = examples["llm_judge_improved"].apply(
```



```
    extract_judge_score
)
```

```
>>> print("Correlation between LLM-as-a-judge and the human raters:")
>>> print(
...     f"{examples['llm_judge_improved_score'].corr(examples['human_score'], method='pearson')}"
... )
```

```
Correlation between LLM-as-a-judge and the human raters:
0.843
```

The correlation was **improved by nearly 30%** with only a few tweaks to the prompt (of which a few percentage points are due to my shameless tip to the LLM, which I hereby declare not legally binding).

Quite impressive! 🍌

Let's display a few errors of our LLM judge to analyse them:

```
errors = pd.concat(
    [
        examples.loc[
            examples["llm_judge_improved_score"] > examples["human_score"]
        ].head(1),
        examples.loc[
            examples["llm_judge_improved_score"] < examples["human_score"]
        ].head(2),
    ]
)

display(
    errors[
        [
            "question",
            "answer",
            "human_score",
            "explanation_1",
```



```

        "llm_judge_improved_score",
        "llm_judge_improved",
    ]
]
)

```

The disagreements are minor: overall, we seem to have reached a good level of performance for our system!

4. How do we take our LLM judge even further?

🎯 **You will never reach 100%:** Let's first note that our human ground truth certainly has some noise, so agreement/correlation will never go up to 100% even with a perfect LLM judge.

🗨️ **Provide a reference:** If you had access to a reference answer for each question, you should definitely give this to the Judge LLM in its prompt to get better results!

📺 **Provide few-shot examples:** adding some few-shot examples of questions and ground truth evaluations in the prompt can improve the results. *(I tried it here, it did not improve results in this case so I skipped it, but it could work for your dataset!)*

✚ **Additive scale:** When the judgement can be split into atomic criteria, using an additive scale can further improve results: see below 🙌

```

ADDITIVE_PROMPT = """
(...)
- Award 1 point if the answer is related to the question.
- Give 1 additional point if the answer is clear and precise.
- Provide 1 further point if the answer is true.
- One final point should be awarded if the answer provides additional resources to support
...
"""

```

Implement with structured generation:



Using **structured generation**, you can configure the LLM judge to directly provide its output as a JSON with fields Evaluation and Total rating, which makes parsing easier : see our [structured generation](#) cookbook to learn more!

Conclusion

That’s all for today, congrats for following along! 🎉

I’ll have to leave you, some weirdos are banging on my door, claiming they have come on behalf of Mixtral to collect H100s. 🤨

`</>` [Update](#) on GitHub

← RAG

Evaluating AI Search Engines with `judges` - the open-source library for LLM-as-a-judge evaluators →

Evaluation